

C++ 自学手册：第 7 章 函数 ——C++ 的编程模块

一、核心知识框架

本章围绕**函数**这一 C++ 编程的核心模块展开，主要内容包括：

- 函数的基本使用（定义、原型、调用）
- 函数参数与按值传递
- 函数与数组（一维、二维）
- 函数与 C 风格字符串
- 函数与结构
- 函数与array对象
- 递归
- 函数指针

二、函数的基本知识

函数是完成特定任务的代码块，使用函数可提高代码复用性和可读性。使用函数需完成三个步骤：**定义函数**、**声明原型**、**调用函数**。

1. 函数定义

函数定义包含函数的具体实现，分为**无返回值函数（void 函数）**和**有返回值函数**。

（1）void 函数（无返回值）

语法：

```
void 函数名(参数列表) {  
    语句块;  
    return; // 可选，用于提前结束函数  
}
```

示例：打印指定次数的 "Cheers!"

```
void cheers(int n) {  
    for (int i = 0; i < n; i++) {  
        cout << "Cheers! ";  
    }  
    cout << endl;  
}
```

(2) 有返回值函数

语法：

```
返回类型 函数名(参数列表) {  
    语句块;  
    return 返回值; // 返回值类型需与声明一致  
}
```

示例：计算数字的立方

```
double cube(double x) {  
    return x * x * x; // 返回double类型值  
}
```

2. 函数原型

函数原型是函数的声明，告知编译器函数的返回类型、参数类型和数量，确保调用时类型匹配。

(1) 语法

```
返回类型 函数名(参数类型列表); // 可省略参数名
```

示例：

```
void cheers(int); // 原型，省略参数名
double cube(double x); // 原型，包含参数名（可选）
```

(2) 作用

- 编译器据此检查函数调用的参数数量和类型是否正确。
- 允许函数定义放在调用之后（如main()之后），不影响编译。

3. 函数调用

通过函数名和参数列表调用函数，根据函数是否有返回值，调用方式略有不同。

(1) 调用 void 函数

直接作为语句调用：

```
cheers(5); // 调用cheers函数，参数为5
```

(2) 调用有返回值函数

可将返回值赋给变量或用于表达式：

```
double volume = cube(2.5); // 将返回值赋给变量
cout << "立方值: " << cube(3) << endl; // 直接使用返回值
```

三、函数参数与按值传递

C++ 默认按值传递参数：函数接收实参的副本，对形参的修改不影响实参。

1. 形参和实参

- **实参**：调用函数时传递的值（如cheers(5)中的5）。
- **形参**：函数定义中接收实参的变量（如void cheers(int n)中的n）。

2. 按值传递的特点

- 函数内操作的是实参的副本，实参本身不受影响。
- 示例：

```
void increment(int x) {  
    x++; // 仅修改形参x的副本  
}  
int main() {  
    int a = 3;  
    increment(a);  
    cout << a; // 输出3，实参a未被修改  
    return 0;  
}
```

3. 多个参数

函数可接收多个参数，参数间用逗号分隔。

示例：计算两个数的调和平均数

```
double harmonic_mean(double x, double y) {  
    return 2.0 * x * y / (x + y);  
}  
// 调用  
double result = harmonic_mean(4, 6); // 返回4.8
```

四、函数与数组

数组作为参数时，实际传递的是数组首元素的地址（指针），函数可直接操作原始数组。

1. 数组作为函数参数

(1) 语法

函数原型需声明数组类型和长度参数（数组长度需显式传递，因数组名仅为指针）：

```
// 原型：arr为数组指针，n为长度
int sum_arr(int arr[], int n);
```

(2) 示例：计算数组总和

```
int sum_arr(int arr[], int n) {
    int total = 0;
    for (int i = 0; i < n; i++) {
        total += arr[i]; // 用下标访问数组元素
    }
    return total;
}
// 调用
int cookies[] = {1, 2, 4, 8};
int total = sum_arr(cookies, 4); // 结果为15
```

2. 用 const 保护数组

若函数无需修改数组，可在参数前加const，防止意外修改：

```
void show_array(const int arr[], int n) { // arr指向常量，不可修改
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
}
```

3. 数组区间传递

通过两个指针指定数组的起始和结束（超尾位置），无需传递长度：

```
// 计算[begin, end)区间内元素的和
int sum_range(const int* begin, const int* end) {
    int total = 0;
```

```
for (const int* pt = begin; pt != end; pt++) {  
    total += *pt; // 用指针访问元素  
}  
return total;  
}  
// 调用：计算前3个元素的和  
int sum = sum_range(cookies, cookies + 3); // 1+2+4=7
```

4. 二维数组作为参数

二维数组参数需指定第二维的长度（列数），第一维长度可省略：

```
// 计算3行4列二维数组的总和  
int sum_2d(int arr[][4], int rows) {  
    int total = 0;  
    for (int r = 0; r < rows; r++) {  
        for (int c = 0; c < 4; c++) {  
            total += arr[r][c];  
        }  
    }  
    return total;  
}  
// 调用  
int data[3][4] = {{1,2,3,4}, {5,6,7,8}, {9,10,11,12}};  
int total = sum_2d(data, 3); // 结果为78
```

五、函数与 C 风格字符串

C 风格字符串以空字符 `'\0'` 结尾，传递时只需传递首地址，函数可通过 `'\0'` 判断结束。

1. 字符串作为参数

示例：统计字符在字符串中出现的次数

```
unsigned int count_char(const char* str, char ch) {  
    unsigned int count = 0;  
    while (*str) { // 当*str不是'\0'时循环
```

```
if (*str == ch) count++;
str++; // 指针移动到下一个字符
}
return count;
}
// 调用
char str[] = "minimum";
int m_count = count_char(str, 'm'); // 结果为3
```

2. 返回字符串的函数

函数无法直接返回字符串，但可返回动态分配的字符串地址（需手动释放内存）：

```
char* build_str(char c, int n) {
    char* pstr = new char[n + 1]; // 预留空字符位置
    pstr[n] = '\0'; // 结尾加空字符
    for (int i = 0; i < n; i++) {
        pstr[i] = c;
    }
    return pstr;
}
// 调用
char* stars = build_str('*', 5); // 返回"*****"
delete[] stars; // 释放内存
```

六、函数与结构

结构可按值传递（传递副本）或传递地址（传递指针），操作方式与基本类型类似。

1. 按值传递结构

示例：计算两个时间结构的总和

```
struct travel_time {
    int hours;
    int mins;
};
```

```

travel_time sum_time(travel_time t1, travel_time t2) {
    travel_time total;
    total.mins = (t1.mins + t2.mins) % 60;
    total.hours = t1.hours + t2.hours + (t1.mins + t2.mins) / 60;
    return total;
}
// 调用
travel_time day1 = {3, 45};
travel_time day2 = {2, 30};
travel_time total = sum_time(day1, day2); // 结果：6小时15分钟

```

2. 传递结构地址

通过指针传递结构，效率更高（无需复制整个结构），需用->访问成员：

```

void show_time(const travel_time* t) { // 用const保护结构
    cout << t->hours << "小时" << t->mins << "分钟" << endl;
}
// 调用
show_time(&total); // 传递结构地址

```

七、函数与 array 对象（C++11）

`std::array`是模板类，可像基本类型一样按值传递，或传递地址。

示例：操作季度开支的array对象

```

#include <array>
const int Seasons = 4;
using namespace std;
// 填充array对象
void fill_array(array<double, Seasons>* pa) {
    for (int i = 0; i < Seasons; i++) {
        cin >> (*pa)[i]; // 注意括号：(*pa)是array对象
    }
}
// 显示array对象
void show_array(array<double, Seasons> da) {

```



```
for (double exp : da) {  
    cout << exp << " ";  
}  
}
```

八、递归

函数可调用自身，称为递归。递归需包含**终止条件**，否则会无限循环。

1. 基本递归示例：倒计时

```
void countdown(int n) {  
    cout << "Counting down: " << n << endl;  
    if (n > 0) {  
        countdown(n - 1); // 递归调用  
    }  
    cout << n << ": Kaboom!" << endl;  
}  
// 调用  
countdown(3);  
// 输出：  
// Counting down: 3  
// Counting down: 2  
// Counting down: 1  
// Counting down: 0  
// 0: Kaboom!  
// 1: Kaboom!  
// 2: Kaboom!  
// 3: Kaboom!
```

2. 递归的特点

- 每级递归都有独立的变量（如示例中的n）。
- 适用于分治问题（如二分查找、阶乘计算），但层级过深可能导致栈溢出。

九、函数指针

函数指针存储函数的地址，可作为参数传递，实现灵活调用不同函数。

1. 声明函数指针

语法：返回类型 (*指针名)(参数类型列表)

```
double (*pf)(double, double); // pf指向接收两个double、返回double的函数
```

2. 赋值与调用

```
// 定义两个函数
double add(double x, double y) { return x + y; }
double multiply(double x, double y) { return x * y; }
int main() {
    double (*pf)(double, double) = add; // 指向add函数
    cout << pf(2, 3) << endl; // 调用add，输出5
    pf = multiply; // 指向multiply函数
    cout << pf(2, 3) << endl; // 调用multiply，输出6
    return 0;
}
```

3. 函数指针作为参数

示例：通过函数指针灵活选择计算方式

```
// 接收两个数和一个函数指针，返回函数计算结果
double calculate(double x, double y, double (*op)(double, double)) {
    return op(x, y);
}
// 调用
double sum = calculate(2, 3, add); // 5
double product = calculate(2, 3, multiply); // 6
```

4. 函数指针数组

存储多个函数指针，可循环调用不同函数：

```
// 数组存储3个函数指针
double (*ops[3])(double, double) = {add, multiply, subtract};
// 循环调用
for (int i = 0; i < 3; i++) {
    cout << ops[i](2, 3) << endl; // 依次调用add、multiply、subtract
}
```

十、关键注意事项

1. **函数原型必须与定义一致**：返回类型、参数数量和类型需完全匹配。
2. **数组参数需显式传递长度**：因数组名是指针，函数无法通过sizeof获取长度。
3. **const 的使用**：对不修改的参数（如数组、结构）加const，提高安全性。
4. **动态内存管理**：函数返回动态分配的内存（如字符串）时，调用者需用delete释放。
5. **递归终止条件**：确保递归在有限步骤内结束，避免栈溢出。

十一、自测练习

1. 编写函数，接收一个int数组和长度，返回数组中偶数的个数。
2. 用递归计算n的阶乘 ($n! = n \times (n-1) \times \dots \times 1$)。
3. 定义函数指针数组，包含加、减、乘、除四个函数，循环调用并输出结果。
4. 编写函数，接收二维数组（3行3列），返回对角线元素的和。

通过本章学习，可掌握函数的各种用法，实现代码的模块化和灵活性，为复杂程序设计奠定基础。

（注：文档部分内容可能由 AI 生成）